

Modulo III - Pipeline Base de Deep Learning

Proyecto: Riesgo crediticio con PyTorch

Entregable: Código completo en PDF, texto seleccionable y legible por IA

Dataset: data/prestamos.csv

Target: loan_status (0 = perfil riesgoso/no aprobado, 1 = perfil apto/aprobado)

Resumen para evaluacion

Este documento acompaña el repositorio del checkpoint. Incluye la estructura del proyecto, la documentación y el código fuente completo del pipeline de entrenamiento y validación en PyTorch.

El pipeline implementa detección automática de dispositivo, semillas de reproducibilidad, preprocesamiento tabular, Dataset/DataLoader, una arquitectura base con nn.Module, training loop con zero_grad, backward y Adam, validación separada, tracking de loss y Accuracy por época.

README.md

```
# Pipeline Base de Deep Learning - Riesgo Crediticio

## Objetivo

Este checkpoint implementa un pipeline base de entrenamiento y validación en PyTorch para un problema de clasificación binaria usando el dataset `prestamos.csv`.

La variable objetivo es `loan_status`:

- `1`: crédito aprobado / perfil apto.
- `0`: crédito no aprobado / perfil riesgoso.

El foco de esta entrega no es construir el modelo más complejo, sino demostrar una infraestructura clara, reproducible y funcional para entrenar y validar un clasificador base.

## Estructura del repositorio

```text
modulo_iii_pipeline_dl/
├── data/
│ └── prestamos.csv
├── src/
│ └── train.py
├── README.md
└── requirements.txt
```

## Configuración del entorno

Crear un entorno virtual e instalar dependencias:

```bash
pip install -r requirements.txt
```

El script detecta automáticamente el dispositivo de ejecución disponible:

- `cuda`, si hay GPU NVIDIA compatible.
- `mps`, si se ejecuta en Apple Silicon.
- `cpu`, si no hay acelerador disponible.

## Cómo ejecutar el entrenamiento
```

Desde la carpeta `modulo\_iii\_pipeline\_dl`:

```
```bash
python src/train.py
```
```

También se pueden modificar hiperparámetros desde consola:

```
```bash
python src/train.py --epochs 15 --batch-size 256 --learning-rate 0.001
```
```

Arquitectura base

Se utiliza un MLP simple implementado como `nn.Module`:

- Capa lineal de entrada.
- Activación `ReLU`.
- `Dropout` para regularización.
- Segunda capa lineal.
- Activación `ReLU`.
- Capa de salida binaria.

La pérdida utilizada es `BCEWithLogitsLoss`, adecuada para clasificación binaria. El optimizador elegido es `Adam`.

Hiperparámetros iniciales

- `learning\_rate`: `0.001`
- `batch\_size`: `128`
- `epochs`: `10`
- `optimizer`: `Adam`
- `loss`: `BCEWithLogitsLoss`

El learning rate `0.001` se eligió como punto de partida estándar para Adam, ya que suele ofrecer una convergencia estable en redes pequeñas.

Pipeline implementado

El script incluye:

- Fijación de semillas de aleatoriedad.
- Detección automática de dispositivo.
- Carga del dataset.
- Ingeniería de atributos simple.
- Separación train/validación con `train\_test\_split`.
- Preprocesamiento de variables numéricas y categóricas.
- Construcción de `Dataset` y `DataLoader`.
- Definición del clasificador base con `nn.Module`.
- Training loop explícito:
 - `forward pass`
 - cálculo de pérdida
 - `optimizer.zero\_grad()`
 - `loss.backward()`
 - `optimizer.step()`
- Ciclo de validación con `model.eval()` y `torch.no\_grad()`.
- Tracking de pérdida y Accuracy por época.

Diagrama conceptual del pipeline

```
```mermaid
graph TD
 A["Carga de datos con pandas"] --> B["Ingeniería de atributos"]
 B --> C["Separación train / validación"]
 C --> D["Preprocesamiento numérico y categórico"]
 D --> E["Dataset y DataLoader de PyTorch"]
 E --> F["Modelo base nn.Module"]
 F --> G["Forward pass"]
 G --> H["Cálculo de pérdida BCEWithLogitsLoss"]
 H --> I["optimizer.zero_grad"]
 I --> J["loss.backward"]
 J --> K["optimizer.step con Adam"]
 K --> L["Evaluación en validación"]
```
```

```
M --> N["Tracking de loss y Accuracy por época"]
```

```
...
```

Resultados esperados

Durante las épocas, el script imprime:

```
- `train_loss`  
- `train_accuracy`  
- `val_loss`  
- `val_accuracy`
```

Además, genera el archivo:

```
```text  
training_history.csv
```
```

Este archivo permite revisar la evolución de la pérdida y la métrica de desempeño por época.

Interpretación de la curva de pérdida

Si el pipeline aprende correctamente, se espera que la pérdida de entrenamiento disminuya a medida que avanzan las épocas. La pérdida de validación también debería bajar o mantenerse estable. Si la pérdida de entrenamiento baja pero la de validación sube, eso puede indicar overfitting.

Este primer checkpoint funciona como infraestructura base para próximos modelos más complejos.

requirements.txt

```
numpy>=1.24  
pandas>=2.0  
scikit-learn>=1.3  
torch>=2.0
```

src/train.py

```
"""
Pipeline base de entrenamiento y validacion en PyTorch.

Dataset: prestamos.csv
Objetivo: clasificacion binaria de loan_status.

Interpretacion de la variable objetivo:
- 1: credito aprobado / perfil apto.
- 0: credito no aprobado / perfil riesgoso.
"""

from __future__ import annotations

import argparse
import random
from pathlib import Path

import numpy as np
import pandas as pd
import torch
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from torch import nn
from torch.utils.data import DataLoader, Dataset

RANDOM_STATE = 42

def set_seed(seed: int = RANDOM_STATE) -> None:
    """Fija semillas para mejorar la reproducibilidad del experimento."""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

def get_device() -> torch.device:
    """Detecta automaticamente GPU CUDA, MPS de Apple Silicon o CPU."""
    if torch.cuda.is_available():
        return torch.device("cuda")
    if hasattr(torch.backends, "mps") and torch.backends.mps.is_available():
        return torch.device("mps")
    return torch.device("cpu")

class LoanDataset(Dataset):
    """Dataset tabular para entregar tensores al DataLoader."""

    def __init__(self, features: np.ndarray, labels: np.ndarray) -> None:
        self.features = torch.tensor(features, dtype=torch.float32)
        self.labels = torch.tensor(labels, dtype=torch.float32).view(-1, 1)

    def __len__(self) -> int:
        return len(self.labels)

    def __getitem__(self, idx: int) -> tuple[torch.Tensor, torch.Tensor]:
        return self.features[idx], self.labels[idx]
```

```

class LoanClassifier(nn.Module):
    """MLP pequeno para clasificacion binaria tabular."""

    def __init__(self, input_dim: int) -> None:
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 64),
            nn.ReLU(),
            nn.Dropout(0.20),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Linear(32, 1),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x)

def build_features(df: pd.DataFrame) -> pd.DataFrame:
    """Crea variables simples para enriquecer la senal del dataset."""
    df = df.copy()
    df["income_log"] = np.log1p(df["person_income"])
    df["loan_amount_log"] = np.log1p(df["loan_amnt"])
    df["credit_history_age_ratio"] = (
        df["cb_person_cred_hist_length"] / (df["person_age"] + 1)
    )
    df["high_interest_rate"] = (
        df["loan_int_rate"] >= df["loan_int_rate"].quantile(0.75)
    ).astype(int)
    return df

def load_and_preprocess(data_path: Path) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]:
    """Carga datos, separa train/validacion y aplica preprocessing tabular."""
    df = pd.read_csv(data_path)
    df = build_features(df)

    target = "loan_status"
    X = df.drop(columns=[target])
    y = df[target].astype(np.float32).values

    numeric_features = X.select_dtypes(include=np.number).columns.tolist()
    categorical_features = X.select_dtypes(exclude=np.number).columns.tolist()

    numeric_transformer = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler()),
        ]
    )
    categorical_transformer = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("encoder", OneHotEncoder(handle_unknown="ignore", sparse_output=False)),
        ]
    )
    preprocessor = ColumnTransformer(
        transformers=[
            ("num", numeric_transformer, numeric_features),
            ("cat", categorical_transformer, categorical_features),
        ]
    )

    X_train, X_val, y_train, y_val = train_test_split(
        X,
        y,
        test_size=0.20,
        random_state=RANDOM_STATE,
        stratify=y,
    )

    X_train_processed = preprocessor.fit_transform(X_train).astype(np.float32)
    X_val_processed = preprocessor.transform(X_val).astype(np.float32)

```

```

    return X_train_processed, X_val_processed, y_train, y_val

def train_one_epoch(
    model: nn.Module,
    dataloader: DataLoader,
    criterion: nn.Module,
    optimizer: torch.optim.Optimizer,
    device: torch.device,
) -> tuple[float, float]:
    """Ejecuta una epoca de entrenamiento con forward, loss, backward y Adam."""
    model.train()
    losses: list[float] = []
    predictions: list[int] = []
    targets: list[int] = []

    for X_batch, y_batch in dataloader:
        X_batch = X_batch.to(device)
        y_batch = y_batch.to(device)

        optimizer.zero_grad()
        logits = model(X_batch)
        loss = criterion(logits, y_batch)
        loss.backward()
        optimizer.step()

        losses.append(loss.item())
        probs = torch.sigmoid(logits)
        preds = (probs >= 0.50).int().detach().cpu().numpy().ravel()
        predictions.extend(preds.tolist())
        targets.extend(y_batch.detach().cpu().numpy().ravel().astype(int).tolist())

    return float(np.mean(losses)), accuracy_score(targets, predictions)

@torch.no_grad()
def evaluate(
    model: nn.Module,
    dataloader: DataLoader,
    criterion: nn.Module,
    device: torch.device,
) -> tuple[float, float]:
    """Evalua el modelo en datos no vistos sin calcular gradientes."""
    model.eval()
    losses: list[float] = []
    predictions: list[int] = []
    targets: list[int] = []

    for X_batch, y_batch in dataloader:
        X_batch = X_batch.to(device)
        y_batch = y_batch.to(device)

        logits = model(X_batch)
        loss = criterion(logits, y_batch)

        losses.append(loss.item())
        probs = torch.sigmoid(logits)
        preds = (probs >= 0.50).int().detach().cpu().numpy().ravel()
        predictions.extend(preds.tolist())
        targets.extend(y_batch.detach().cpu().numpy().ravel().astype(int).tolist())

    return float(np.mean(losses)), accuracy_score(targets, predictions)

def run_training(
    data_path: Path,
    epochs: int,
    batch_size: int,
    learning_rate: float,
) -> None:
    """Orquesta el pipeline completo de entrenamiento y validacion."""
    set_seed(RANDOM_STATE)

```

```

device = get_device()
print(f"Dispositivo seleccionado: {device}")
print(f"Version de PyTorch: {torch.__version__}")

X_train, X_val, y_train, y_val = load_and_preprocess(data_path)
train_dataset = LoanDataset(X_train, y_train)
val_dataset = LoanDataset(X_val, y_val)

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)

model = LoanClassifier(input_dim=X_train.shape[1]).to(device)
criterion = nn.BCEWithLogitsLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)

print(f"Arquitectura base:\n{model}")
print(f"Learning rate: {learning_rate}")
print("-" * 72)

history = []
for epoch in range(1, epochs + 1):
    train_loss, train_acc = train_one_epoch(
        model, train_loader, criterion, optimizer, device
    )
    val_loss, val_acc = evaluate(model, val_loader, criterion, device)
    history.append(
        {
            "epoch": epoch,
            "train_loss": train_loss,
            "train_accuracy": train_acc,
            "val_loss": val_loss,
            "val_accuracy": val_acc,
        }
    )

    print(
        f"Epoch {epoch:02d}/{epochs} | "
        f"train_loss={train_loss:.4f} | train_acc={train_acc:.4f} | "
        f"val_loss={val_loss:.4f} | val_acc={val_acc:.4f}"
    )

history_df = pd.DataFrame(history)
output_path = Path("training_history.csv")
history_df.to_csv(output_path, index=False)
print("-" * 72)
print(f"Historial guardado en: {output_path.resolve()}")

first_loss = history_df["val_loss"].iloc[0]
last_loss = history_df["val_loss"].iloc[-1]
if last_loss < first_loss:
    print("Interpretacion: la perdida de validacion bajo durante el entrenamiento.")
else:
    print(
        "Interpretacion: la perdida de validacion no bajo; conviene revisar "
        "hiperparametros, arquitectura o numero de epocas."
    )

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="Pipeline base de deep learning para clasificacion crediticia."
    )
    parser.add_argument(
        "--data-path",
        type=Path,
        default=Path("data/prestamos.csv"),
        help="Ruta al dataset prestamos.csv.",
    )
    parser.add_argument("--epochs", type=int, default=10)
    parser.add_argument("--batch-size", type=int, default=128)
    parser.add_argument("--learning-rate", type=float, default=1e-3)
    return parser.parse_args()

```

```
if __name__ == "__main__":  
    args = parse_args()  
    run_training(  
        data_path=args.data_path,  
        epochs=args.epochs,  
        batch_size=args.batch_size,  
        learning_rate=args.learning_rate,  
    )
```